

SPOTIFY PROJECT

Final Technical Report

COURSE	STACK	DELIVERY
Web Programming - S2026	React 19 + Django 5.2	Docker + PWA + Realtime

Repository implementation, tests, deployment configuration, and assignment coverage were reviewed for this final report.

Student identities are confirmed by the team; responsibility descriptions below remain based on repository evidence.

1. Team and responsibilities

Contributor / student ID	Repository evidence	Primary responsibility (repository-based)
Mahyar Rezaei Pourkhalili 402106037	Mahyar	Operational backend, payments, notifications, support, PWA, Docker, realtime group listening and delivery tooling.
Ali Mirzaei Feyzabadi 402106672	Alimirzaei84	Music domain, player, albums, playlists, artist works, playback integration and sample data.
Mohammad Parsa Hassanzadeh 402106423	JimmyNotron	Application foundation, authentication, profiles, subscriptions, shared frontend structure and tests.

Working conventions

- Feature-oriented React modules and Django apps split by business domain.
- REST payloads use snake_case at the API boundary and map to camelCase UI models.
- JWT authentication plus server-side role and subscription permissions protect sensitive operations.
- Schema migrations accompany model changes; focused tests cover API and frontend service contracts.

Maintainability

Rendering, HTTP services, serializers, permissions, domain services and persistence are separated. Shared entitlement selectors avoid duplicating access rules. Environment variables configure PostgreSQL, Redis, CORS and Zarinpal; Docker defines the reproducible deployment topology.

2. Architecture

Layer	Technology	Responsibilities
Client	React 19, TypeScript, Vite	Responsive pages, player state, forms, PWA shell and WebSocket client.
API	Django 5.2, DRF	Authentication, validation, CRUD, roles, subscriptions, reports and payment orchestration.
Realtime	Django Channels, Redis	Temporary listening rooms, membership and synchronized playback events.
Data	PostgreSQL	Users, preferences, catalog, streams, tickets, notifications, transactions and rooms.
Edge	Nginx	Static frontend delivery and reverse proxying for HTTP, media and WebSockets.

Request flow

Browser traffic enters Nginx. Static navigation is served by React; /api routes reach Django/Daphne; /ws connections upgrade to Channels. Django authenticates JWTs, applies role and plan permissions, validates serializers and persists through PostgreSQL. Redis carries realtime messages.

Deployment

compose.yaml defines frontend Nginx, Django/Daphne, PostgreSQL and Redis. The backend entrypoint waits for dependencies, applies migrations and collects static assets. Volumes preserve database and uploaded-media data.

PWA

The web manifest, icons and registered service worker provide installability and offline shell startup. Authenticated API responses are deliberately excluded from caches.

3. Domain model and access rules

Aggregate	Key relationships
Identity	User 1:1 UserPreference; verified artist 1:1 ArtistProfile; User 1:N ArtistApplication reviews.
Subscriptions	SubscriptionPlan 1:N UserSubscription and SubscriptionOrder; orders retain gateway state.
Music	Artist(User) 1:N Album and Song; Album 1:N Song; Playlist M:N Song through PlaylistSong; Song stores collaborators.
Listening	User and Song create Stream/RecentlyPlayed rows; temporary ListeningGroup has members and current Song.
Operations	Tickets/messages, notifications and artist payout/report rows link to users by role.

Server-enforced integrity

- Playlist creation is rejected after the active plan's Free/Silver/Gold limit.
- Stream creation is rejected at the daily limit before a Stream row or play-count increment occurs.
- Early-access albums and songs are hidden from public catalog, search, chart, playlist, history, download and realtime paths unless entitled.
- Artist aggregate stream statistics are visible only to Gold subscribers, administrators, or the artist owner.
- Only verified artists may publish; ownership protects catalog mutations; audio validation caps supported uploads at 50 MB.
- Support and admin roles manage tickets; only administrators change plan prices and financial/audit state.

4. Implemented feature coverage

Requirement group	Implementation
Authentication	Listener/artist registration, login, JWT refresh/logout, privacy acknowledgement and password reset.
Discovery and profiles	Home, search, recent playlists, albums/singles, profiles, follows and verification badge.
Playback	Play/pause, seek, queue, shuffle, repeat, lyrics, stream accounting and server-side daily limits.
Artist tools	Verified-artist album/song CRUD, collaborators, covers, metadata, lyrics, audio and Gold-gated statistics.
Plans and payments	Free/Silver/Gold entitlements, durations, admin prices and Zarinpal sandbox request/verification.
Operations	Backend notifications, support chat, artist review, API-backed audit UI and monthly artist reporting.
Optional	Temporary invite-code listening groups with synchronized state and last-member cleanup.

Validation completed

- Frontend: 32 test files, 163 tests passed; production TypeScript/Vite build passed.
- Backend dependencies, including Channels, Daphne and pytest-asyncio, are installed in the project virtual environment.
- Backend: 223 tests passed against isolated SQLite, including Channels realtime tests and entitlement regressions.
- Django system check and migration consistency check pass with no model changes missing.
- Docker Compose configuration resolves frontend, backend, PostgreSQL and Redis services; runtime smoke testing still requires a running Docker Desktop daemon.

5. AI-assisted development and final status

AI assistance was used as an implementation and review aid: mapping the assignment to repository features, generating repetitive serializer/service/test scaffolding, reviewing authorization boundaries, identifying local-browser fallbacks, correcting aggregation, preparing Docker/PWA configuration and drafting this report.

Strengths

- Fast cross-layer checks across React types, API payloads, serializers and routes.
- Focused regression tests for permissions, limits and frontend/backend contracts.
- Identification of superficially complete features that still lacked persistence or backend enforcement.

Safeguards and limits

- Generated code was verified through repository searches, builds, focused tests and complete suites.
- External payment behavior and container startup still depend on Zarinpal and a running Docker environment.
- Student identities are confirmed above; responsibility descriptions should be adjusted if the team uses a different agreed division.

Final submission checklist

- Run `docker compose up --build` on a machine with Docker Desktop running and verify health, payment callback and WebSocket flow.
- Retain the green frontend/backend logs and review this report against institutional academic-integrity rules.
- Do not commit local secrets, database files, caches, coverage output or dependency directories.